

Chapter 44

Capstone: An End-to-End Data Science Project

Walk one business question through the whole Part IV lifecycle — frame, prepare, model, evaluate, and communicate — without skipping a stage · combine a model's probability with the actual size of what's at stake, and see why ranking by probability alone can miss most of the value · wrap a...

Approved chapter · v1

Contents

Chapter 44. Capstone: An End-to-End Data Science Project

Why this matters

In plain English

44.1 The question, and the plan

44.2 Refitting the model (a recap, not a re-teach)

44.3 From probability to rupees

44.4 Building the call list: probability isn't the same as value

44.5 Explaining two accounts

44.6 One function, not six steps

44.7 The one-page summary

44.8 The Part IV toolkit, applied

Common mistakes and how to spot them

In the real world: what happened to the call list

Tools

The project: your own end-to-end pass

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 44. Capstone: An End-to-End Data Science Project

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: walk one business question through the whole Part IV lifecycle — frame, prepare, model, evaluate, and communicate — without skipping a stage · combine a model's probability with the actual size of what's at stake, and see why ranking by probability alone can miss most of the value · wrap a multi-step analysis into one reusable function, the first step toward something you could actually hand off or schedule · write a one-page, non-technical summary that a leader can act on, and know what belongs in it and what doesn't.

Before you start: this chapter assumes everything from Chapters 35–43. It does not re-teach any of it; it uses it. If a step feels unfamiliar, the chapter says exactly which earlier chapter taught it, so you can go back rather than guess.

Time needed: 4–6 hours.

Tools: Python 3 with scikit-learn and shap — nothing new.

Practice data: Riverstone's customer accounts (Chapter 37); the same 5,000 accounts, the same churn target, the same pipeline. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Every chapter in Part IV taught one skill in isolation: a split, a model, a metric, an explanation. Real work rarely arrives in those tidy pieces. Someone asks a vague question, the data needs assembling, a dozen small decisions have to be made in order, and at the end somebody who has never heard of a confusion matrix needs to know what to do differently on Monday morning.

This chapter is one worked pass through that whole arc, start to finish, on a single question: ***which of Riverstone's at-risk accounts should the sales team call this month, and why those and not others?*** Nothing here is a new technique. What's new is the order, the joins between stages, and one habit most tutorials skip entirely: turning a probability into a decision that accounts for how much is actually at stake.

In plain English

Imagine a doctor doing a batch of check-ups with a limited number of follow-up appointments to hand out.

A model that says "this patient has a 40% chance of a problem" is useful, but it's not yet a plan. Two patients might both sit at 40%: one whose problem, if missed, is minor, and one whose problem, if missed, is serious. Handing out follow-ups by risk percentage alone would treat those two patients identically. Handing them out by **risk times what's actually at stake** would not. That's the shift this chapter makes to Riverstone's churn model: a probability becomes a *decision* only once it's weighed against the size of the account behind it.

44.1 The question, and the plan

"Which at-risk accounts should the sales team call this month?"

That's a business request, not yet a data science question. Chapter 36 taught the five things to pin down before touching data; here they are for this project:

Decision	This project
Unit	One customer account
Target	Will this account place no order in 2025? (Chapter 37's <code>churned_2025</code>)
Prediction moment	As of 31 December 2024, using only that year's data
Success measure	Rupees of revenue protected, against the cost of the calls made
Capacity	The sales team can make 40 retention calls this month

The plan, in the order the rest of the chapter follows it:

1. **Refit** Chapter 37's churn model exactly as before — nothing new here, just a reminder of what it does.
 2. **Turn each account's probability into a rupee value:** how much revenue is genuinely at risk, not just how likely the account is to leave.
 3. **Build the call list** from that value, not from probability alone, and see how much it differs.
 4. **Explain** two accounts on the list well enough that a rep could open the call with something specific to say.
 5. **Wrap steps 1–3 into one function**, so the whole analysis can be re-run with one line instead of retyped by hand.
 6. **Write the one-page summary** a non-technical leader actually receives.
-

44.2 Refitting the model (a recap, not a re-teach)

```
import warnings

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train, valid = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)

prepare = ColumnTransformer(
    [
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
        (
            "num",
            Pipeline(
                [
                    ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                    ("scale", StandardScaler()),
                ]
            ),
            NUMS,
        ),
    ],
)

model = Pipeline(
    [
        ("prepare", prepare),
        (
            "model",
            HistGradientBoostingClassifier(
                learning_rate=0.05,
                max_depth=3,
                min_samples_leaf=40,
            )
        ),
    ]
)
```

```

        max_iter=300,
        random_state=37,
    ),
],
)
model.fit(train[CATS + NUMS], train["churned_2025"])
valid = valid.copy()
valid["p_churn"] = model.predict_proba(valid[CATS + NUMS])[:, 1]
print(f"{len(train):,} training accounts, {len(valid):,} validation accounts")
print(
    f"validation AUC: {roc_auc_score(valid['churned_2025'], valid['p_churn']):.3f}    "
    f"(Chapter 37's tuned boosting model, same pipeline and settings)"
)

```

```

3,000 training accounts, 1,000 validation accounts
validation AUC: 0.788    (Chapter 37's tuned boosting model, same pipeline and settings)

```

How it works: every line here is Chapter 37's, unchanged — the same `CATS` and `NUMS`, the same stratified split with `random_state=37`, the same `ColumnTransformer` (fill blanks, one-hot encode, scale), the same tuned `HistGradientBoostingClassifier`. If any of that reads unfamiliar, that's Chapter 37's material, not this chapter's; go back there before continuing. The only line worth pausing on is the last one: `valid["p_churn"] = ...` **adds a new column** to the validation table holding each account's predicted probability, so every later step can look the account up by row and immediately see its score alongside its other columns — a small habit that makes everything downstream far easier to read.

44.3 From probability to rupees

A probability alone doesn't say what to do with it. Riverstone loses far more by ignoring a large wholesale account with a modest churn risk than by ignoring a small retailer who's almost certain to leave. The fix is to multiply the model's probability by what the account is actually worth:

```

CALL_COST = (
    1200 # rupees: a retention call plus follow-up, roughly Chapter 39's lead-work cost
)
MARGIN = 0.15 # gross margin, same assumption Chapter 39 used

valid["value_at_risk"] = valid["p_churn"] * valid["revenue_2024"] * MARGIN
print(
    valid[["account_id", "segment", "revenue_2024", "p_churn", "value_at_risk"]]
    .head(3)
    .to_string(index=False)
)
print(
    f"\ntotal value at risk across all {len(valid):,} validation accounts: "
    f"₹{valid['value_at_risk'].sum():,.0f}"
)

```

account_id	segment	revenue_2024	p_churn	value_at_risk
7881	Retail	80400.0	0.058636	707.153017
8530	Hospitality	185500.0	0.053669	1493.335409
5016	Retail	264800.0	0.126821	5037.319525

total value at risk across all 1,000 validation accounts: ₹3,093,334

The plan in words, before the arithmetic below: for each account, take its churn probability, multiply by its 2024 revenue, then multiply by the share of that revenue Riverstone actually keeps as profit (its margin) — the result is the rupee value genuinely at risk if nothing is done.

value at risk = $P(\text{churn}) \times 2024 \text{ revenue} \times \text{margin}$

How it works:

- `CALL_COST` and `MARGIN` are named constants, not numbers buried in a formula — exactly Chapter 39's habit of writing business assumptions as variables a reader can see, question, and change.
- `valid["p_churn"] * valid["revenue_2024"] * MARGIN` multiplies three columns together, row by row: pandas does this for all 1,000 accounts in one line, the same **element-by-element arithmetic** Chapter 35 introduced with NumPy arrays.

Setting	What it means in plain words	Value used here	What happens if you change it
<code>CALL_COST</code>	Rupees to make one retention call and follow up	1,200	Higher: fewer accounts clear the break-even bar; lower: more do
<code>MARGIN</code>	Share of revenue that's actual profit, not just turnover	0.15 (15%)	Higher: every account's value at risk rises, so more calls look worthwhile; lower: fewer do

Reading the output. The third listed account, at ₹264,800 in 2024 revenue and a 12.7% churn probability, has ₹5,037 at risk — modest on its own, but multiply this across 1,000 accounts and Riverstone has just over ₹3 million of revenue genuinely on the line this year.

44.4 Building the call list: probability isn't the same as value

With only 40 calls available, which 40 accounts?

```

CAPACITY = 40 # the sales team's capacity: 40 retention calls this month

by_value = valid.sort_values("value_at_risk", ascending=False).head(CAPACITY)
by_probability = valid.sort_values("p_churn", ascending=False).head(CAPACITY)
overlap = len(set(by_value["account_id"]) & set(by_probability["account_id"]))

print(
    f"ranking by value at risk: top {CAPACITY} accounts hold "
    f"₹{by_value['value_at_risk'].sum():,.0f} of value at risk"
)
print(
    f"ranking by probability alone: top {CAPACITY} accounts hold "
    f"₹{by_probability['value_at_risk'].sum():,.0f} of value at risk"
)
print(f"the two lists share only {overlap} of {CAPACITY} accounts")

```

```

ranking by value at risk: top 40 accounts hold ₹1,156,056 of value at risk
ranking by probability alone: top 40 accounts hold ₹374,379 of value at risk
the two lists share only 10 of 40 accounts

```

Reading it. Ranking by value at risk captures **₹1,156,056**; ranking by churn probability alone captures only **₹374,379** — less than a third as much, from the same model and the same 40-account budget. The two lists barely overlap: only 10 of the 40 accounts appear on both. This isn't a modeling improvement; the model's predictions haven't changed at all. It's a **decision** improvement: a probability-only ranking spends the sales team's scarce time on accounts that are *likely* to leave regardless of how little they'd cost Riverstone, while missing large accounts whose departure would be expensive even at a lower probability. This is the single most important habit in this chapter: **a score is not a decision until it's weighed against what's actually at stake**, exactly as Chapter 39's cost-based thresholds argued, now applied where the value itself varies row by row instead of being the same for everyone.

Predict before you run this. Before reading the next section, guess: will the highest-value account on the list have a *high* or a *moderate* churn probability? Then look at block D's output.

44.5 Explaining two accounts

A call list with no reason attached is hard for a rep to use. Chapter 39 taught SHAP for exactly this:

```

import shap

feature_names = model.named_steps["prepare"].get_feature_names_out()
X_valid_prepared = model.named_steps["prepare"].transform(valid[CATS + NUMS])
explainer = shap.TreeExplainer(model.named_steps["model"])
shap_values = explainer.shap_values(X_valid_prepared)

top_account = by_value.iloc[0]
row_position = valid.index.get_loc(top_account.name)
contributions = pd.Series(shap_values[row_position], index=feature_names)
print(
    f"account {top_account['account_id']} ({top_account['segment']}, "
    f"₹{top_account['revenue_2024']:,.0f} revenue): predicted churn probability "
    f"{top_account['p_churn']:.1%}"
)
print(contributions.abs().sort_values(ascending=False).head(4).index.tolist())
print("largest pushes (positive = toward churning):")
print(
    contributions.reindex(
        contributions.abs().sort_values(ascending=False).head(4).index
    )
    .round(3)
    .to_string()
)

```

```

account 7001 (Wholesale, ₹7,966,300 revenue): predicted churn probability 8.4%
['num__late_payment_days', 'num__avg_discount_pct', 'num__units_2024', 'num__complaints_2024']
largest pushes (positive = toward churning):
num__late_payment_days    0.922
num__avg_discount_pct     0.760
num__units_2024           -0.673
num__complaints_2024      0.484

```

Reading it. The account at the very top of the call list — Wholesale, ₹7.97 million in 2024 revenue — has a churn probability of only 8.4%, well below what a probability-only ranking would flag, and it's on the list purely because 8.4% of nearly ₹8 million is still a large number. Its largest pushes toward churning are late payments and a high average discount; its units bought pulls the other way, toward staying. That's a specific opening line for the account manager: *"Their payment timing has slipped and their discount has crept up — worth checking whether something's changed on their side before it becomes a bigger problem."* An explanation doesn't need to be exhaustive to be useful; it needs to give the rep one true, specific thing to ask about.

44.6 One function, not six steps

Everything so far has been six separate steps, run once, by hand. The next stage of any real project is making it **repeatable**: one function that takes the raw data and settings, and returns the answer.

The plan in words: take the raw accounts table and three settings (the cost of a call, the margin, and how many calls the team can make), and return three things: the fitted model, the call list, and how good the model is — in one call, with nothing left to retype.

```

def build_call_list(accounts_df, call_cost=1200, margin=0.15, capacity=40):
    accounts_df = accounts_df.copy()
    accounts_df["rep_id"] = accounts_df["rep_id"].astype(str)
    rest_split, test_split = train_test_split(
        accounts_df,
        test_size=0.2,
        random_state=37,
        stratify=accounts_df["churned_2025"],
    )
    train_split, valid_split = train_test_split(
        rest_split, test_size=0.25, random_state=37, stratify=rest_split["churned_2025"]
    )
    prepare_step = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                        ("scale", StandardScaler()),
                    ]
                ),
                NUMS,
            ),
        ],
    )
    fitted_model = Pipeline(
        [
            ("prepare", prepare_step),
            (
                "model",
                HistGradientBoostingClassifier(
                    learning_rate=0.05,
                    max_depth=3,
                    min_samples_leaf=40,
                    max_iter=300,
                    random_state=37,
                ),
            ),
        ],
    )
    fitted_model.fit(train_split[CATS + NUMS], train_split["churned_2025"])
    valid_split = valid_split.copy()
    valid_split["p_churn"] = fitted_model.predict_proba(valid_split[CATS + NUMS])[:, 1]
    valid_split["value_at_risk"] = (
        valid_split["p_churn"] * valid_split["revenue_2024"] * margin
    )
    call_list = valid_split.sort_values("value_at_risk", ascending=False).head(capacity)
    auc = roc_auc_score(valid_split["churned_2025"], valid_split["p_churn"])
    return fitted_model, call_list, auc

_, default_list, default_auc = build_call_list(accounts)
print(
    f"default run: AUC {default_auc:.3f}, call list of {len(default_list)} accounts, "
    f"₹{default_list['value_at_risk'].sum():.0f} value at risk"
)

```

```
default run: AUC 0.788, call list of 40 accounts, ₹1,156,056 value at risk
```

How it works, line by line:

- `def build_call_list(accounts_df, call_cost=1200, margin=0.15, capacity=40):` declares a function with three settings given **default values**, so it can be called with no arguments at all (using the defaults) or with any of them overridden.
- The next eight lines are sections 44.2's model-fitting steps, unchanged, just renamed with a `_split` suffix so they don't collide with the `train / valid` names already used earlier in this chapter — a real concern once code moves from a notebook's scattered cells into a function of its own.
- `valid_split["value_at_risk"] = ...` repeats section 44.3's calculation, now using the function's own `margin` argument instead of a fixed constant, which is what makes it changeable from outside.
- `.sort_values("value_at_risk", ascending=False).head(capacity)` repeats section 44.4's ranking, using the function's `capacity` argument.
- `return fitted_model, call_list, auc` sends back three separate values in one line; calling code unpacks them in the same order, as the next line shows.

```
for capacity_setting in [20, 40, 80]:
    _, call_list, _ = build_call_list(accounts, capacity=capacity_setting)
    print(
        f"capacity={capacity_setting:>3}: total value at risk on the list "
        f"₹{call_list['value_at_risk'].sum():>10,.0f}"
    )
```

```
capacity= 20: total value at risk on the list ₹    861,828
capacity= 40: total value at risk on the list ₹ 1,156,056
capacity= 80: total value at risk on the list ₹ 1,519,267
```

Reading it. Calling the same function three times, changing only `capacity`, shows the trade-off directly: doubling the team's capacity from 40 to 80 calls roughly captures more value (₹1.52 million against ₹1.16 million), but not twice as much, because the highest-value accounts were already included at capacity 40 — the marginal 41st-through-80th accounts are worth less each. This is the kind of question a sales head actually asks ("what if we gave you ten more hours a week?"), and now it's one line to answer rather than a rebuilt analysis.

Changed-line exercise. In block F's loop, change `[20, 40, 80]` to `[40, 200]`. Before running it, predict whether the value captured at capacity 200 will be closer to double or closer to the same as capacity 80's result, and why — then check.

44.7 The one-page summary

Everything above is for you. What a leader receives is shorter, and every number in it must trace back to something already computed:

```

final_model, final_call_list, final_auc = build_call_list(accounts, capacity=40)
segment_counts = final_call_list["segment"].value_counts()
print(f"model quality: AUC {final_auc:.3f}")
print(
    f"call list: {len(final_call_list)} accounts, "
    f"₹{final_call_list['value_at_risk'].sum():.0f} total value at risk"
)
print("by segment:", segment_counts.to_dict())
print(
    f"median revenue on the list: ₹{final_call_list['revenue_2024'].median():.0f}    "
    f"(all validation accounts: ₹{valid['revenue_2024'].median():.0f})"
)

```

```

model quality: AUC 0.788
call list: 40 accounts, ₹1,156,056 total value at risk
by segment: {'Wholesale': 17, 'Retail': 13, 'Hospitality': 10}
median revenue on the list: ₹770,300    (all validation accounts: ₹222,850)

```

That table of numbers becomes a page of prose, not a slide of metrics:

To: Anita Rao, Sales Head From: Analytics Re: February 2026 retention call list

The churn model correctly separates accounts likely to leave from accounts likely to stay about 79% of the time when compared pairwise — a solid, usable model, not a perfect one.

We've built this month's 40-account call list by combining each account's churn risk with its actual revenue, not risk alone. Together, these 40 accounts represent about **₹1.16 million** of revenue we'd expect to lose if nothing changes. Ranking by risk alone, ignoring account size, would have pointed the team at accounts worth only **₹374,000** combined — the same number of calls, less than a third of the value protected.

The list skews toward larger accounts: its median revenue is **₹770,300**, well above the typical validation account's **₹222,850**. That's expected and intended — this list exists specifically to catch valuable accounts a simple risk ranking would miss — but it does mean the team's smaller, high-risk accounts need a separate, lower-touch process (an email campaign, say) rather than a personal call.

By segment, the list is 17 Wholesale, 13 Retail, and 10 Hospitality accounts. Each account manager will get their own accounts flagged with the two or three factors driving that account's risk, so every call opens with something specific rather than a generic "how's everything going?"

What this list is not: proof that calling these accounts will retain them, only an estimate of what's at risk if nobody does. Whether calls actually change the outcome is a question for a proper test (Chapter 30), which we'd recommend running on a sample before scaling this to every account every month.

Notice what's absent: no AUC in the second paragraph (it's mentioned once, plainly, and never again), no code, no confusion matrix, no mention of gradient boosting or SHAP by name. The reader needs to know what to do and how confident to be in it, not how it was built. The **limitations paragraph is not optional** — a summary that only reports the upside is the kind of overclaiming this book has argued against since Chapter 1.

44.8 The Part IV toolkit, applied

Every chapter in this part shows up somewhere in the six steps above:

Chapter	What it contributed here
35	The dot products and gradients inside the model; the log-loss thinking behind evaluating probabilities
36	The framing table, the split, the pipeline, the leakage discipline in CATS / NUMS
37	The tuned gradient-boosting model itself, unchanged
38	The idea that a business decision (which accounts to prioritize) doesn't always need a predictive target — useful for the low-touch process the summary recommends for smaller accounts
39	Cost-based thresholds, generalized here to a value that varies per row, and SHAP for individual explanations
40	The habit of testing an assumption (capacity, cost) with a real re-run rather than guessing at the effect
41	Not used directly here, but the same "recompute honestly, don't assume" discipline applied to text elsewhere
42	The same ranking-and-evaluation mindset (precision at a fixed list length) reused for a call list instead of a product list
43	The comparison discipline: before reaching for anything fancier, this project stayed with Chapter 37's gradient-boosting model because nothing in this part gave a reason to change it

Nothing here required a new algorithm. It required using the ones already learned, in the right order, without skipping the step that turns a number into a decision.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Ranking purely by model probability	Large, moderately-at-risk accounts never make the list	Multiply probability by what's actually at stake
Treating the call list as proof calls work	"We'll definitely save this revenue"	State plainly that this is an estimate of risk, not a measured effect (Chapter 30)
A summary full of model metrics	The reader can't find the actual recommendation	Lead with the decision; mention the metric once
No cost or capacity constraint	A list of thousands, useless to a team with 40 hours	Always tie the list to a real capacity or a real cost
Copy-pasting the same six steps for every re-run	Small differences creep in between runs	Wrap the steps in one function, as section 44.6 does
No limitations section	Leadership overtrusts the list	Always state what the model does not prove

In the real world: what happened to the call list

Anita takes February's list to her three account managers. Farah, given her share of the Wholesale accounts, questions one immediately: a large account with a modest churn probability but a high value-at-risk score. "They're not going anywhere," she says. "They just haven't ordered as much this quarter because of a plant shutdown I already know about."

That's not a flaw in the list; it's exactly why the list is a starting point for a conversation, not a verdict. The model has no way to know about a plant shutdown it was never shown. Farah's local knowledge and the model's account-wide pattern-matching are doing different jobs, and the useful outcome is combining them — she still makes the call, but she opens it differently than the model's SHAP explanation would have suggested. Two months later, when Meera reviews which accounts on the list actually reordered, that's the first real evidence of whether calling changes anything at all, and it's the beginning of the proper test the summary recommended.

Tools

Nothing new. Every tool used in this chapter — scikit-learn's pipeline and gradient boosting, shap's `TreeExplainer` — was introduced and explained in Chapters 36, 37, and 39.

The project: your own end-to-end pass

Goal: run this chapter's six-step lifecycle on a question of your own, ending in a one-page summary someone outside data science could act on.

Steps:

1. Pick a question with a real decision behind it and a real constraint (a budget, a headcount, a number of hours) — not just "predict X."
 2. Frame it with Chapter 36's table: unit, target, prediction moment, success measure, capacity.
 3. Reuse a model you've already built in this part rather than building a new one from scratch.
 4. Turn its output into a value, not just a probability or a score.
 5. Wrap the steps into one function, and show one "what if we changed the capacity/cost" result with real numbers.
 6. Explain two individual cases well enough that someone could act on them immediately.
 7. Write the one-page summary, including a limitations paragraph that isn't an afterthought.
-

You've got it when...

- ☐ I can name, for any model I've built, what decision it actually feeds — not just what it predicts.

- ☐ I never rank purely by probability when the things being ranked have different value.
 - ☐ I can wrap a multi-step analysis into one function with sensible defaults.
 - ☐ I write a one-page summary that leads with the decision, not the metric, and always states a limitation.
 - ☐ I can point to which earlier chapter taught any given step in my own pipeline.
-

Recap

- A **probability is not a decision**; multiplying it by what's genuinely at stake (a value at risk) can completely reorder a priority list, as it did here: three times the value protected, from the same model.
 - **Capacity and cost constraints** belong in the analysis from the start, not bolted on afterward.
 - **Wrapping repeated steps in one function** is the first move toward something reusable, schedulable, or handed to someone else.
 - A **one-page summary** leads with the decision, states the model's quality once, and always includes what the analysis does not prove.
 - An end-to-end project is not a new skill; it's the discipline of using Chapters 35–43's tools in the right order and not skipping the step that turns a number into an action.
-

Practice exercises

1. (*hand*) An account has a 30% churn probability and ₹500,000 in 2024 revenue. At a 15% margin, what's its value at risk? Is it worth a ₹1,200 call by the per-account break-even rule from section 44.3?
 2. Rebuild the call list with `MARGIN` set to 0.25 instead of 0.15. Does the same set of 40 accounts appear, or does the ranking change? Why might margin differ by segment in reality, and what would that do to the list?
 3. Using `build_call_list`, produce a call list with `capacity=40` and `call_cost=15000`. How many of the 40 accounts now fail the per-account break-even test from section 44.3, even though they're still in the top 40 by value at risk? What does that tell you about the difference between *ranking* by value and *filtering* by break-even?
 4. Write the one-paragraph "what this list is not" limitation section for your own project from this chapter's project brief. What is the one thing a reader might wrongly conclude from your summary if you left that paragraph out?
-

Key terms

value at risk · break-even per row · capacity constraint · reusable pipeline function · default argument · executive summary · limitations statement

(All terms are defined in the Glossary, Appendix A.)

Where this leads

This chapter closes Part IV. Part V moves from single models to the systems and habits that keep them honest and useful over time: business metrics that connect a model's output to money (Chapter 45), experiments that prove whether an intervention like a retention call actually works (Chapter 30, referenced throughout this chapter), and deploying and monitoring models once they leave a notebook for good (Chapter 52). The call list this chapter built is exactly the kind of thing those later chapters take further: from a one-off analysis to a system that runs every month, is tested rather than assumed to work, and is watched for when it starts to drift.

Answers to practice exercises

1. value at risk = $0.30 \times ₹500,000 \times 0.15 = ₹22,500$. Since ₹22,500 is well above the ₹1,200 call cost, yes — by a wide margin, this account clears the per-account break-even test from section 44.3.

2.

```
_, list_margin_25, _ = build_call_list(accounts, margin=0.25)
_, list_margin_15, _ = build_call_list(accounts, margin=0.15)
same_accounts = set(list_margin_25["account_id"]) == set(list_margin_15["account_id"])
print(f"same 40 accounts at margin 0.25 and 0.15? {same_accounts}")
```

```
same 40 accounts at margin 0.25 and 0.15? True
```

The exact same 40 accounts appear either way. Margin multiplies every account's value at risk by the same fixed number, so it stretches or shrinks the whole scale uniformly without changing anyone's position relative to anyone else — the **ranking** is unaffected, even though the raw rupee figures reported in the summary would change. In reality, margin genuinely differs by segment (Wholesale typically runs thinner margins than Hospitality, say): using one fixed margin for every account, as this chapter does for simplicity, would then distort the ranking, favoring segments whose margin is understated and underweighting those whose margin is overstated. A more careful version would use each account's own segment-level margin rather than one number for everyone.

3.

```
_, list_high_cost, _ = build_call_list(accounts, call_cost=15000, capacity=40)
below_breakeven = (list_high_cost["value_at_risk"] < 15000).sum()
print(
    f"accounts in the top 40 that fail a ₹15,000 break-even test: {below_breakeven} of 40"
)
print(
    f"lowest value at risk on this list: ₹{list_high_cost['value_at_risk'].min():,.0f}"
)
```

```
accounts in the top 40 that fail a ₹15,000 break-even test: 13 of 40
lowest value at risk on this list: ₹11,770
```

At a ₹15,000 call cost, 13 of the top 40 accounts by value at risk would actually **lose** Riverstone money if called — their value at risk doesn't clear the higher cost, even though they still rank in the top 40 overall. This is the key distinction the exercise is after: **ranking** by value at risk answers "which accounts matter most, relative to each other?", while **filtering** by break-even answers a different question, "which accounts are worth acting on at all, given what it costs to act?" A sensible call list, once the cost is this high, should apply both: rank first, then drop anyone below break-even, rather than assuming the top N by rank are automatically worth calling.

4. (No single correct answer — this asks you to write your own project's limitations paragraph.)

Whatever the specific project, the paragraph should distinguish between what the analysis shows (a risk estimate, a ranking, a pattern in past data) and what it does not prove (that acting on it changes the outcome, that the pattern will hold as conditions change, that no other factor explains what the model found). Without it, the most common wrong conclusion a reader draws is treating a **correlational risk score** as if it were a **measured effect of taking action** — exactly the gap this chapter's own summary calls out in its final paragraph, and exactly what a proper experiment (Chapter 30) is needed to close.